

Reviewer Pack — Fundamental Group Rank Inverse Proportional to Resolution Pr...

Entry 2905aa033f8a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 05:05:53 UTC

Fundamental Group Rank Inverse Proportional to Resolution Proof Size for Tseitin Formulas

Entry ID: 2905aa033f8a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 05:05:53 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Fundamental Groups of Graphs) **Field B** (complexity object): Resolution Proof Size for Tseitin Formulas

Statement:

For a Tseitin formula derived from a connected expander graph with m edges and n vertices, the resolution proof size is at least $2^{c \cdot (m - n + 1)}$ for some absolute constant $c > 0$. The exponent equals the rank of the fundamental group π_1 of the graph.

Rationale (proposer's reasoning):

Expander graphs' high connectivity implies nontrivial π_1 structure, which may encode combinatorial obstructions to resolution proofs. The exponential dependency suggests that topological complexity directly limits proof efficiency, offering a novel algebraic-topological barrier to resolution.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8f9a3369884e2ec8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_expander_graph(n, m):
    graph = [[] for _ in range(n)]
    edges_added = 0
    while edges_added < m:
        u, v = random.sample(range(n), 2)
        if u != v and v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
            edges_added += 1
    return graph

def find_fundamental_group_rank(graph):
    n = len(graph)
    visited = [False] * n
    rank = 0

    def dfs(node, parent):
        nonlocal rank
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                dfs(neighbor, node)
            elif neighbor != parent:
                rank += 1

    for i in range(n):
        if not visited[i]:
            dfs(i, -1)

    return rank

def generate_tseitin_formula(graph):
    n = len(graph)
    literals = [f'x{i}' for i in range(n)]
    clauses = []
```

```

# Add clauses for each edge
for u, v in [(i, j) for i in range(n) for j in graph[i]]:
    clauses.append([f'~{literals[u]}', f'{literals[v]}'])
    clauses.append([f'{literals[u]}', f'~{literals[v]}'])

# Add clauses to ensure each literal appears exactly once
for i in range(n):
    clauses.append([f'{literals[i]}'])
    clauses.append([f'~{literals[i]}'])

return literals, clauses

def dpll_with_clause_learning(clauses, literals):
    def is_satisfiable():
        assignment = {}

        def backtrack(level):
            if level == len(literals):
                return True
            literal = literals[level]
            for value in [True, False]:
                assignment[literal] = value
                if all([not clause or any(assignment.get(lit, not value) for lit in clause) for clause in clauses]):
                    if backtrack(level + 1):
                        return True
            del assignment[literal]
            return False

        return backtrack(0)

    return is_satisfiable()

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = int(1.5 * n)
    graph = generate_expander_graph(n, m)
    rank = find_fundamental_group_rank(graph)
    literals, clauses = generate_tseitin_formula(graph)

    proof_size = dpll_with_clause_learning(clauses, literals)

    if not proof_size:
        return {
            "metric_name": "resolution_proof_size",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "DPLL did not find a proof"
        }

    c = 0.3
    expected_size = 2 ** (c * (m - n + 1))

    return {

```

```

        "metric_name": "resolution_proof_size",
        "metric_value": proof_size,
        "instances_tested": 1,
        "conjecture_holds": proof_size >= expected_size,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 230, 7)) # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std_dev = math.sqrt(sum((r['metric_value'] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 593, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 631, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 677, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 727, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 773, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 821, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 877, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}
TRIAL: {"seed": 929, "metric_name": "resolution_proof_size", "metric_value": 0, "instances_tested": 1}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_32d3e164.py", line 134, in <module>
    first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
    ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_32d3e164.py", line 134, in <genexpr>
    first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
    ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing required trials; insufficient data to evaluate support fraction or counterexamples | next: Fix the test code's KeyError handling and re-run with complete seed coverage

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113352
2	propose	ollama_remote	qwen3:8b	0	0	129286
3	preregistration	ollama_remote	qwen3:8b	0	0	24232
4	novelty	ollama_remote	qwen3:8b	0	0	20651
5	novelty	ollama_remote	qwen3:8b	0	0	15185
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48466
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12482
8	judge	ollama_remote	qwen3:8b	0	0	18747

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 382402 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/2905aa033f8a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2905aa033f8a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2905aa033f8a.tar.gz (if generated)